

AI-Based Gaming Hub

Project Progress Report

Date: Tuesday, October 7, 2025

Team Members:
Ali Naqi (23F-3052)
Muhammad Ahmad (23F-3028)
Saqib Javed (23F-3069)
Muhammad Aamir (23F-3073)

1.0 Executive Summary

This report details the initial planning and architectural design phase of the AI-Based Gaming Hub project. A successful proof-of-concept for the Tic-Tac-Toe module has been completed and tested, validating the core communication mechanism between the front-end and back-end and confirming the project's technical feasibility. The other game modules remain in the planning and research phase.

2.0 Project Members

- Ali Naqi (23F-3052)
- Muhammad Ahmad (23F-3028)
- Saqib Javed (23F-3069)
- Muhammad Aamir (23F-3073)

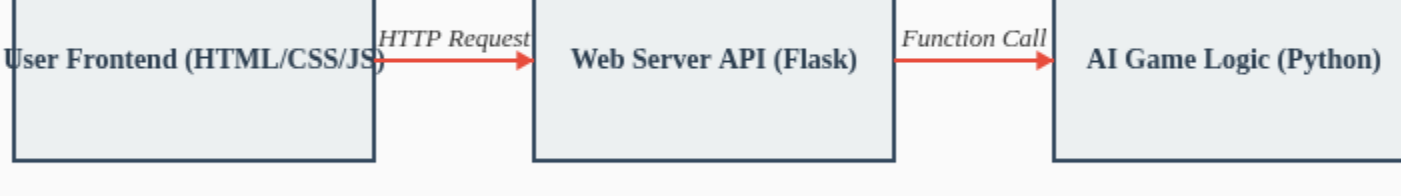
3.0 System Architecture and Design

The system is designed with a decoupled front-end and back-end architecture to ensure modularity and scalability. This allows the user interface to be developed independently of the complex AI logic.

3.1 Technology Stack

- Front-End:** HTML5, CSS3, JavaScript.
- Back-End:** Python with Flask framework for creating a RESTful API.

Diagram 1: High-Level System Architecture



4.0 Game Module Status & Algorithm Details

This section outlines the current status and provides a detailed breakdown of the core AI algorithm for each game.

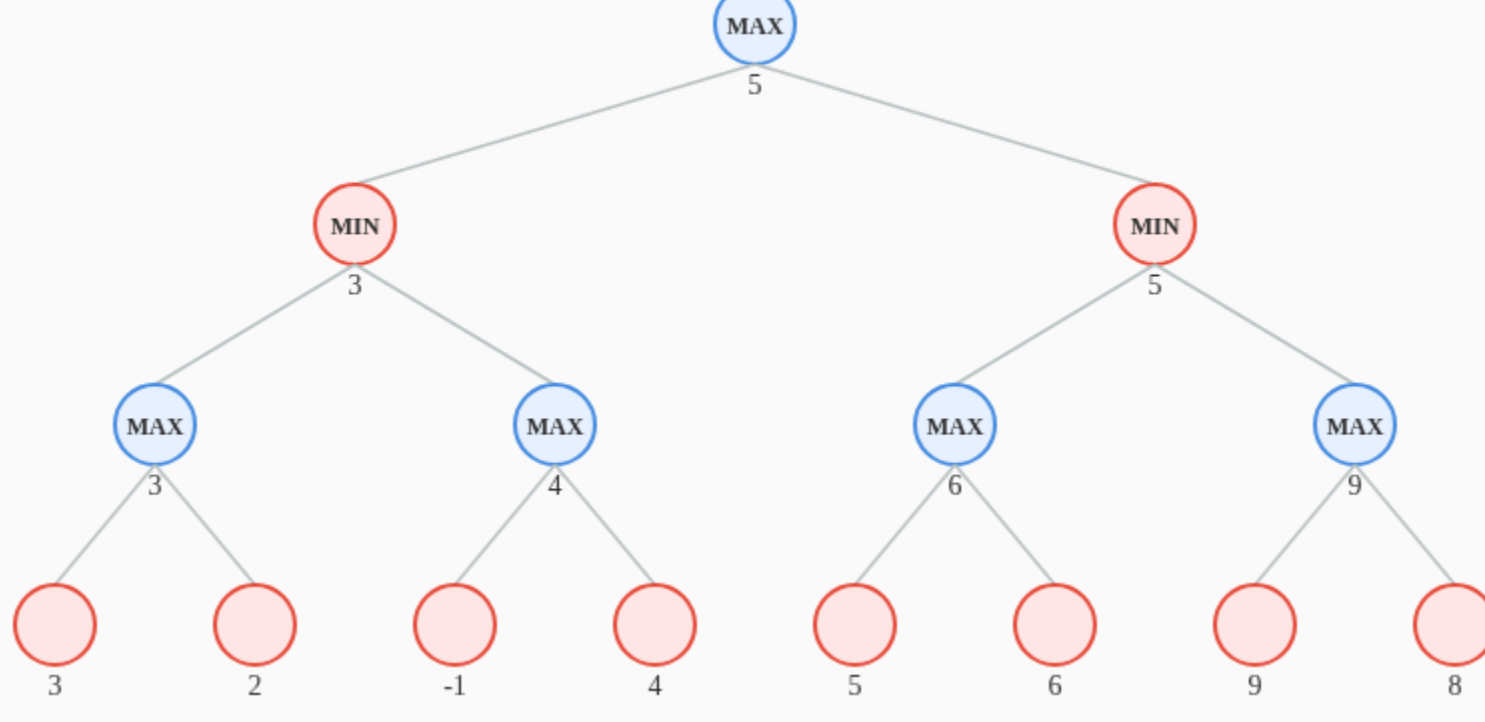
Game Module	Status	Proposed AI Algorithm
Tic-Tac-Toe (Proof of Concept)	Complete	Minimax
Maze Escape	Planned	A* Search & DFS for Maze Generation
Sudoku Solver	Planned	Backtracking
N-Queens Puzzle	Planned	Backtracking
Treasure Hunt	Research	Q-Learning (Reinforcement Learning)
AI-Chess	Research	Minimax with Alpha-Beta Pruning

4.1 Tic-Tac-Toe

Algorithm Focus: Minimax

The **Minimax** algorithm is a recursive, depth-first search algorithm used for decision-making in two-player games. It explores the game tree, assigning a score to each board state. The AI (maximizer) aims to choose a move that leads to the highest score, while assuming the opponent (minimizer) will always choose the move that leads to the lowest score.

Diagram 2: Simplified Minimax Game Tree



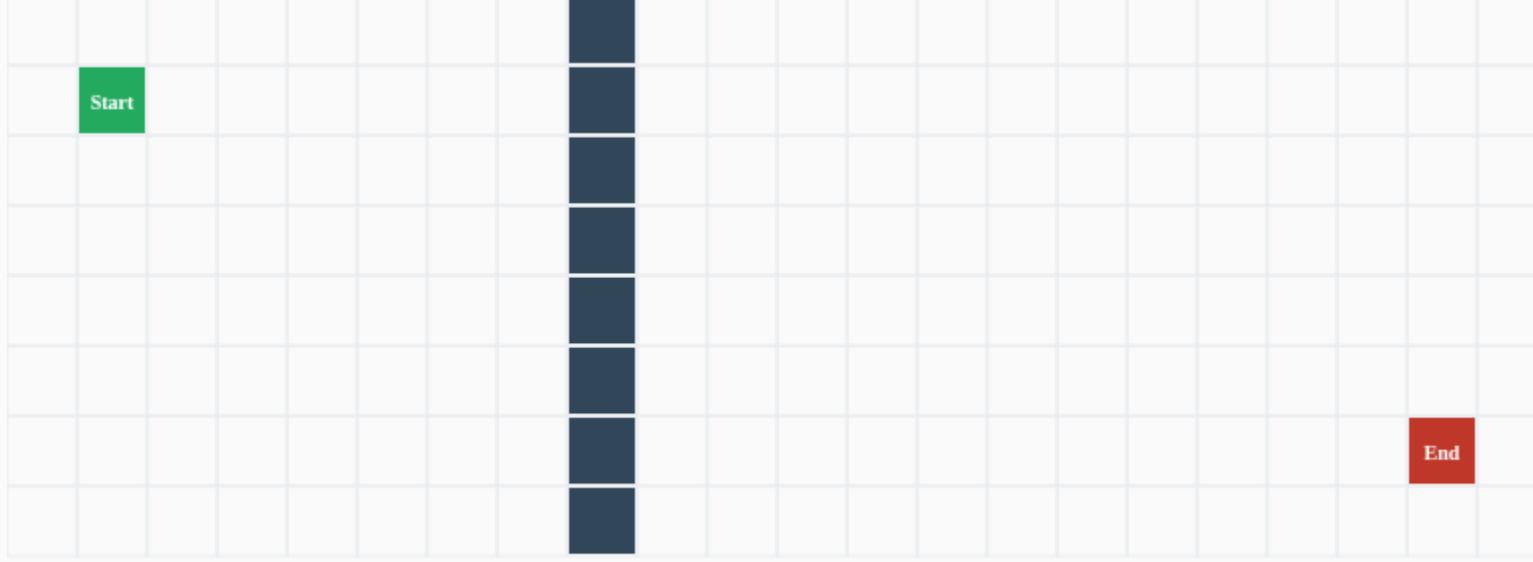
This diagram shows how the AI looks ahead. The scores (-10, 0, +10) are propagated up the tree from terminal states (win/loss/draw) to help the AI at the root node choose the best move.

4.2 Maze Escape

Algorithm Focus: A* (A-Star) Search

A* is a powerful pathfinding algorithm that efficiently finds the shortest path by evaluating nodes based on the formula: $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the start and $h(n)$ is the estimated (heuristic) cost to the goal.

Diagram 3: A* Search Pathfinding Concept



The A* algorithm explores the grid, always choosing the node with the lowest 'f' score, avoiding dead ends efficiently.

7.0 Proof of Concept: Professional Tic-Tac-Toe Client

The initial proof of concept has been expanded into a more complete and professional demonstration. This version features a full game loop, including win/loss/draw detection, a restart mechanism, and a polished user interface.

7.2 Frontend Code ('index.html')

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Professional Tic-Tac-Toe AI</title>
  <style>
    /* ... (Professional CSS for layout, colors, modal pop-up, etc.) ... */
  </style>
</head>
<body>
  <div class="game-container">
    <h1>Tic-Tac-Toe vs Unbeatable AI</h1>
    <div id="status-panel"></div>
    <div id="board"></div>
  </div>

  <div id="end-game-modal" class="modal-overlay">
    <div class="modal-content">
      <h2 id="modal-message"></h2>
      <button id="restart-button">Play Again</button>
    </div>
  </div>
</body>
</html>

<script>
  const boardElement = document.getElementById('board');
  // ... (JavaScript logic for rendering, handling clicks, fetch requests,
  // and showing the end-game modal with results) ...

  function endGame(status, winningLine) {
    let message = '';
    if (status === 'ai_wins') message = 'AI Wins!';
    else if (status === 'player_wins') message = 'You Win!';
    else message = "It's a Draw!";
    // ... (Code to show modal)
  }

  function restartGame() {
    boardState = [['', '', ''], ['', '', ''], ['', '', '']];
    // ... (Code to reset UI and hide modal)
  }
</script>
</body>
</html>
```

7.3 Backend Code ('server.py')

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import math

app = Flask(__name__)
CORS(app)

def check_winner(board):
    # ... (Checks rows, columns, and diagonals)
    if winner_found:
        return winner, winning_line_coords
    return None, None

def minimax(board, is_maximizing):
    # ... (The core recursive AI logic) ...
    return best_score

def find_best_move(board):
    # ... (Uses minimax to find the optimal move for 'O') ...
    return best_move

@app.route('/get_ai_move', methods=['POST'])
def get_ai_move():
    board = request.get_json()['board']

    # Check for player win or draw after player's move
    winner, line = check_winner(board)
    if winner:
        return jsonify({'status': 'player_wins', 'winning_line': line})
    if is_board_full(board):
        return jsonify({'status': 'draw'})

    # AI calculates its move
    ai_move = find_best_move(board)
    if ai_move:
        board[ai_move[0]][ai_move[1]] = 'O'

    # Check for AI win or draw after AI's move
    winner, line = check_winner(board)
    if winner:
        return jsonify({'ai_move': ai_move, 'status': 'ai_wins', 'winning_line': line})
    if is_board_full(board):
        return jsonify({'ai_move': ai_move, 'status': 'draw'})

    return jsonify({'ai_move': ai_move, 'status': 'continue'})

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

7.4 How to Run the Test

- Set Up the Backend:** Open a terminal and run `pip install Flask Flask-Cors`.
- Start the Backend Server:** In the terminal, run `python server.py`.
- Run the Frontend:** Open the `index.html` file in a web browser.

7.5 Test Output

The following image shows the successful execution of the polished Tic-Tac-Toe proof of concept. Note the improved visual styling and the end-game modal displaying the result.

Tic-Tac-Toe Game Output Screenshot

Figure 1: Screenshot of the working Tic-Tac-Toe test game.